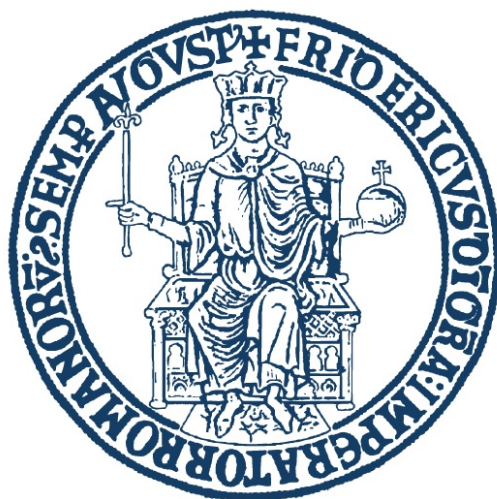




UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II
CORSO DI LAUREA TRIENNALE IN INFORMATICA

Documentazione Hackathon Object Orientation

Mario Majorano	N86005035
Luca Sanselmo	N86005147

Anno Accademico 2024/2025

Indice

1	Definizione del Dominio del Problema	2
1.1	Obiettivi del Sistema	2
1.2	Analisi dei Requisiti	3
1.2.1	Requisiti Informativi	3
1.2.2	Vincoli Individuati	3
1.2.3	Requisiti Funzionali	4
2	Definizione del Modello di Dominio del Problema	6
2.1	Identificazione delle Classi Candidate	6
2.1.1	Classi Indivuate	7
2.1.2	Classi Indivuate — Classi Associative	7
2.2	Individuazione delle Responsabilità	7
2.2.1	Assegnazione delle Responsabilità	7
2.2.2	Class-Responsibility-Collaborator Cards	8
2.3	Identificazione Associazioni ed Attributi	9
2.3.1	Indentificazione Associazioni	10
2.3.2	Identificazione Attributi	11
2.4	Identificazione Generalizzazioni ed Interfacce	12
2.4.1	Generalizzazioni Individuate	12
2.4.2	Vincoli della Generalizzazione	12
2.4.3	Interfacce Individuate	12
2.5	Modello di Dominio del Problema	13
3	Dizionari	14
3.1	Dizionario delle Classi	14
3.2	Dizionario delle Associazioni	15
3.3	Dizionario Classi Associative	16
4	Packages	18
4.1	Package GUI	19
4.2	Package Controller	20
4.3	Package Model	21
4.4	Package DAO	22
5	Dominio della Soluzione	23
5.1	Solution Diagram	23
6	Sequence Diagrams	24
6.1	Creazione di un Hackathon	24
6.2	Pubblicazione di un Progresso	25
7	Manuale per l'uso	26
8	Repository GitHub	44

Introduzione

La presente documentazione ha l'obiettivo di:

1. Descrivere l'analisi effettuata per la **risoluzione** del modello di dominio del problema proposto dal committente
2. Portare il lettore alla comprensione delle **scelte implementative** adottate per la **progettazione software**
3. Rappresentare gli elementi concreti descritti dal contesto del mondo reale trattato tramite la definizione di **classi**, caratterizzate dai relativi **attributi** e **responsabilità**, e delle **associazioni** che intercorrono tra di esse.

1 Definizione del Dominio del Problema

I requisiti e le informazioni tali da costituire il dominio del problema sono specificate all'interno della seguente traccia di esercizio:

Traccia del Problema

Un hackathon, ovvero una "maratona di hacking", è un evento durante il quale team di partecipanti si sfidano per progettare e implementare nuove soluzioni basate su una certa tecnologia o mirate a un certo ambito applicativo.

Ogni hackathon ha un titolo identificativo, si svolge in una certa sede e in un certo intervallo di tempo (solitamente 2 giorni) e ha un organizzatore specifico (registrato alla piattaforma). L'organizzatore seleziona un gruppo di giudici (selezionati tra gli utenti della piattaforma, invitandoli). Infine, l'organizzatore apre le registrazioni, che si chiuderanno 2 giorni prima dell'evento. Ogni evento avrà un numero massimo di iscritti e una dimensione massima del team.

Durante il periodo di registrazione, gli utenti possono registrarsi per l'Hackathon di loro scelta (eventualmente registrandosi sulla piattaforma se non lo hanno già fatto). Una volta iscritti, gli utenti possono formare team. I team diventano definitivi quando si chiudono le iscrizioni. All'inizio dell'hackathon, i giudici pubblicano una descrizione del problema da affrontare.

Durante l'hackathon, i team lavorano separatamente per risolvere il problema e devono caricare periodicamente gli aggiornamenti sui "progressi" sulla piattaforma come documento, che può essere esaminato e commentato dai giudici. Alla fine dell'hackathon, ogni giudice assegna un voto (da 0 a 10) a ciascun team e la piattaforma, dopo aver acquisito tutti i voti, pubblica le classifiche dei team.

1.1 Obiettivi del Sistema

A partire dalla traccia fornita, il sistema da progettare deve consentire:

- La gestione completa del ciclo di vita di un hackathon
- La registrazione e autenticazione degli utenti sulla piattaforma
- La formazione e gestione dei team di partecipanti
- Il caricamento e la valutazione dei documenti prodotti
- La generazione automatica delle classifiche finali

1.2 Analisi dei Requisiti

1.2.1 Requisiti Informativi

Il sistema prevede la realizzazione di una piattaforma atta all'organizzazione e alla gestione di una maratona di hacking in cui team di partecipanti collaborano per sviluppare soluzioni innovative in un tempo limitato, solitamente 48 ore. Ogni hackathon è unico e prevede una serie di caratteristiche ben definite che ne regolano l'organizzazione:

- **Organizzazione dell'Hackathon:**
L'**Hackathon** viene creato dalla figura dell'**Organizzatore**, un **Utente** registrato sulla piattaforma, che ne definisce i dettagli fondamentali: un *titolo* identificativo, la *sede* in cui si svolgerà, la *durata* (tipicamente due giorni), e parametri operativi come il *numero massimo di partecipanti* e la *dimensione massima dei team*.
- **Selezione dei Giudici:**
Una volta configurato l'evento, l'**Organizzatore** seleziona un gruppo di **Giudici** tra gli **Utenti** già presenti nella piattaforma, invitandoli a valutare i progetti presentati dai **Partecipanti**.
- **Registrazione e formazione dei Team:**
Le iscrizioni all'**Hackathon** vengono aperte dall'**Organizzatore** e rimangono disponibili fino a 48 ore prima dell'inizio dell'evento, momento in cui i **Team** formati diventano definitivi. In questo periodo, gli **Utenti** possono registrarsi singolarmente o formare **Team**. Se un **Utente** non è ancora registrato sulla piattaforma, può creare un account in fase di iscrizione.
- **Svolgimento dell'Hackathon**
All'inizio della competizione, i **Giudici** pubblicano la *descrizione del problema* da risolvere, che servirà come base per lo sviluppo dei progetti. Durante l'evento, i **Team** lavorano in modo indipendente ma sono tenuti a caricare periodicamente aggiornamenti sui loro progressi sotto forma di **Documenti** successivamente valutati dai **Giudici**.
- **Valutazione e classifica finale**
Al termine dell'**Hackathon**, ogni **Giudice** assegna un *voto* a ciascun **Team** in base alla qualità della soluzione proposta. La piattaforma raccoglie automaticamente tutti i voti e pubblica una classifica finale resa disponibile a tutti gli **Utenti**.

1.2.2 Vincoli Individuati

Per garantire il corretto funzionamento della sistema, sono state imposte le seguenti regole:

- Un **Utente** può assumere più di un ruolo, ma **non** in una *singola* competizione
- Un **Utente** deve essere registrato all'**Hackathon** prima di poter prendere parte ad un **Team**
- Un **Utente** può unirsi a più **Team**, ma **non** in in *singolo Hackathon*
- Un **Team** può essere formato anche da un solo **Partecipante**
- Un gruppo di **Giudici**, per essere considerato valido, deve essere composto da almeno 2 membri
- Un **Hackathon**, per essere considerato valido, deve prevedere almeno 2 **Team**
- Il *Voto* finale deve essere compreso tra 0 e 10
- Prima di poter dare un *Voto* ad un **Team**, un **Giudice** deve *esaminare* ogni documento prodotto da quel team.
- Un **Partecipante** deve rispettare i vincoli temporali impostegli dall'**Organizzatore** per poter effettuare le operazioni che desidera.
- Per essere valutati correttamente, i **Team** devono produrre almeno 1 **Documento**

Formalizzazione dei Vincoli

$\mathcal{G} = \{g_1, \dots, g_k\}$	$ \mathcal{G} \geq 2$	Insieme dei Giudici
$\mathcal{P} = \{p_1, \dots, p_m\}$		Insieme dei Partecipanti
$\mathcal{O} = \{o_1, \dots, o_n\}$	$ \mathcal{O} = 1$	Insieme degli Organizzatori, 1 per Hackathon
$\mathcal{T} = \{t_0, t_1, \dots, t_z\}$	$ \mathcal{T} \geq 2$	Insieme dei Team
$t_i = \{p_{i,1}, \dots, p_{i, t_i }\} \subseteq \mathcal{P}, \quad t_i \cap t_j = \emptyset \quad \forall i \neq j$		Partecipanti di ogni team, disgiunti tra loro
$voto \in [0, 10]$		

$$\mathcal{H} = \langle \mathcal{P}, \mathcal{O}, \mathcal{G}, \mathcal{T} \rangle \quad \text{Istanza di Hackathon}$$

Convenzioni Notazionali

$p_{i,j}$	Partecipante numero j del team t_i
i	Indice del team, $i \in \{0, \dots, z\}$
j	Posizione del partecipante all'interno del team t_i , $j \in \{1, \dots, t_i \}$

1.2.3 Requisiti Funzionali

Il sistema per la gestione degli hackathon deve soddisfare una serie di funzionalità volte a supportare il ciclo di vita completo degli eventi di programmazione competitiva, dalla creazione dell'evento fino alla pubblicazione delle classifiche finali.

Gestione degli utenti

Registrazione di nuovi utenti sulla piattaforma, specificando:

- Credenziali di accesso
- Dati anagrafici

Autenticazione e gestione del profilo utente.

Gestione degli hackathon

Creazione di un nuovo hackathon da parte di un organizzatore, specificando:

- Titolo identificativo
- Sede dell'evento
- Intervallo temporale (tipicamente 2 giorni)
- Numero massimo di iscritti
- Dimensione massima del team

Visualizzazione e ricerca di hackathon per:

- Titolo
- Sede
- Data
- Organizzatore

Gestione dei giudici

Selezione e invito di giudici tra gli utenti registrati

Pubblicazione della descrizione del problema da affrontare all'inizio dell'hackathon.

Consultazione e commento dei progressi caricati dai team.

Gestione delle registrazioni

Apertura delle registrazioni per un hackathon specifico.

Registrazione dei partecipanti durante il periodo di iscrizione.

Controllo del rispetto del numero massimo di iscritti.

Gestione dei team

Formazione dei team da parte dei partecipanti registrati:

- Creazione di nuovi team
- Adesione ad altri team
- Controllo del rispetto della dimensione massima del team

Finalizzazione automatica dei team alla chiusura delle iscrizioni.

Visualizzazione della composizione dei team.

Gestione dei progressi

Caricamento periodico degli aggiornamenti sui progressi:

- Upload di documenti contenenti lo stato di avanzamento

Consultazione dei progressi da parte dei giudici.

Aggiunta di commenti e feedback da parte dei giudici.

Gestione delle valutazioni

Assegnazione dei voti finali:

- Voto da 0 a 10 per ciascun team da parte di ogni giudice
- Controllo della completezza delle valutazioni
- Calcolo automatico delle medie e classifiche

Pubblicazione delle classifiche finali una volta acquisiti tutti i voti.

Consultazione dei risultati e delle statistiche dell'evento.

2 Definizione del Modello di Dominio del Problema

Definizione: il *Modello di Dominio del Problema* (Domain Model) è un modello che tiene conto della descrizione dei requisiti del problema da risolvere **senza nessun riferimento** ad elementi di quella che sarà la **soluzione** che poi verrà realizzata per **risolvere tale problema**

Per la definizione di tale modello è necessario un processo di **astrazione** composto dalle seguenti fasi:

1. Identificazione di un primo insieme di **Classi Candidate**
2. Aggiungimento di **Associazioni ed Attributi** a tali Classi
3. Identificazione delle **Generalizzazioni**
4. Identificazione delle principali **Responsabilità** di ogni Classe
5. **Iterare** il processo finchè il modello ottenuto non è soddisfacente

2.1 Identificazione delle Classi Candidate

Nella fase di progettazione del modello di dominio del problema, tramite l'applicazione dei concetti generali della modellazione *Object Oriented*, vengono individuate le classi: ad ogni classe corrisponde un'entità del dominio del problema definito.

Definizione: Una *Classe* è un'astrazione ben definita che modella un elemento del dominio del problema.

Una classe per essere considerata tale, deve possedere le seguenti caratteristiche:

- Il nome della Classe ne rispecchia l'intento
- Possiede un insieme ridotto e ben definito di **responsabilità**
- Ha una massima coesione interna

Analisi dei Nomi

Definizione: L'*Analisi dei Nomi* è una tecnica utile a determinare le classi rappresentanti gli elementi del dominio del problema.

Tale processo consiste delle seguenti fasi:

- Analisi della descrizione dei requisiti precedentemente determinati
- Estrarre da tale analisi nomi e predicati nominali:
 - Eliminazione di nomi che
 - * Sono ridondanti
 - * Rappresentano istanze e non classi
 - * Sono vaghi
 - * Corrispondono a classi che non sono necessarie al livello considerato

2.1.1 Classi Individuate

Di seguito l'elenco delle classi individuate, **risultato** della precedente **fase di analisi dei nomi**:

Utente	Team
Rappresenta tutti gli attori del sistema.	Gruppi di lavoro dei partecipanti all' Hackathon.
Partecipante	Documento
Giudice	I documenti prodotti dai team.
Organizzatore	Hackathon
Specializzazioni della classe Utente utili ad individuare il ruolo assunto dagli attori del sistema	Evento principale del sistema.

2.1.2 Classi Individuate — Classi Associative

Sono state individuate inoltre due classi definite *associative* in quanto utili ad identificare una particolare occorrenza di associazione tra classi. In particolare:

- Con l'introduzione della classe **Registrazione** si intende sottolineare l'importanza della relazione che intercorre tra *Hackathon* e *Partecipante*: un'istanza di Registrazione fa riferimento ad un singolo *Hackathon* e ad un singolo *Partecipante*.
- Mentre la classe **Votazione**, assolve il suo ruolo tramite l'introduzione dell'attributo *Voto*, riferito sia al *Team* ricevente che al corrispettivo *Giudice* che lo emette; questo permette quindi di individuare la coppia Giudice-Team e contemporaneamente conservare il valore della votazione.

2.2 Individuazione delle Responsabilità

2.2.1 Assegnazione delle Responsabilità

Definizione: Una *responsabilità* corrisponde a un compito o a un comportamento richiesto al sistema e deve essere attribuita a una specifica classe.

- Ogni **requisito funzionale deve quindi essere associato ad almeno una classe**, anche nel caso in cui la sua realizzazione implichi una **collaborazione tra più classi**. Le responsabilità di una classe devono risultare **chiaramente correlate** tra loro, in modo da garantire coerenza e chiarezza nel modello.
- Se una classe presenta **troppe responsabilità**, è opportuno valutarne la suddivisione in più classi distinte, così da migliorare la manutenibilità e la modularità del sistema. Al contrario, una classe priva di responsabilità concrete può essere considerata **superflua** e quindi rimossa.
- Infine, qualora emerga una responsabilità che non può essere assegnata a nessuna delle classi esistenti, è consigliabile procedere con la **creazione di una nuova classe** dedicata, al fine di mantenere una corretta distribuzione dei compiti all'interno dell'architettura.

La fase di Assegnazione delle Responsabilità è stata effettuata tramite la creazione di una **Class-Responsibility-Collaborator** per ogni classe individuata, in cui sono elencate rispettivamente le responsabilità e i collaboratori che vi interagiscono nel contesto di quest'ultima.

2.2.2 Class-Responsibility-Collaborator Cards

Definizione: Le *Class-Responsibility-Collaborator* (CRC) sono strumenti di progettazione che utilizzano semplici schede di carta per rappresentare le classi, le loro responsabilità e le associazioni necessarie per svolgere tali responsabilità.

Nella presente fase, per ogni classe identificata, viene posto il suo nome su una scheda (Card) e man mano che vengono individuati attributi e responsabilità, questi vengono ripostati su di esse.

Utente

Responsabilità	Collaboratori
• Registrazione alla piattaforma	• Hackathon

Partecipante

Responsabilità	Collaboratori
• Iscrizione all'Hackathon • Creazione Team • Entra a far parte del Team	• Team • Hackathon • Documento

Giudice

Responsabilità	Collaboratori
• Pubblicazione del problema • Aggiunta di un commento al Documento • Assegnazione del voto	• Team • Documento • Hackathon

Organizzatore

Responsabilità	Collaboratori
• Invitare i Giudici • Aprire le Registrazioni ad un Hackathon	• Giudice • Hackathon

Team

Responsabilità	Collaboratori
• Pubblicazione dei Documenti	• Partecipante • Documento • Giudice

Documento

Responsabilità	Collaboratori
• $\emptyset$	• $\emptyset$

Hackathon

Responsabilità	Collaboratori
• $\emptyset$	• $\emptyset$

2.3 Identificazione Associazioni ed Attributi

- Il processo di Identificazione Associazioni ed Attributi inizia a partire dalle **classi centrali**, considerate fondamentali per il funzionamento del sistema. Per ciascuna di esse vengono identificati i **dati principali** e le **relazioni essenziali** con le altre classi.
- Successivamente si passa alle **classi secondarie**, completando la struttura generale del modello. Durante lo sviluppo si mantiene un approccio orientato alla **semplicità**, evitando un eccesso di attributi o associazioni.
- Infine, si privilegia una progettazione in cui le **classi gestiscono poche informazioni** e presentano **basse dipendenze reciproche**, garantendo così un sistema più chiaro, modulare ed efficiente.

2.3.1 Identificazione Associazioni

Definizione: Un' *Associazione* è una relazione strutturale che lega due o più classi, specificando che gli oggetti istanze di tali classi possono essere connessi o comunicare tra loro.

Ognuna di esse definisce il **tipo** e la **molteplicità** del legame tra le entità coinvolte. Di seguito sono elencate le associazioni che intercorrono tra le classi del sistema:

Registrazione	Pubblicazione
Partecipazione	Votazione
Competizione	Selezione
Organizzazione	Esaminazione

Le suddette associazioni sono state descritte nel dettaglio nel [Dizionario delle Associazioni](#)

Identificazione Associazioni — Note

Per rendere il sistema il più possibile aderente al contesto del mondo reale descritto, le associazioni

- **Partecipazione** (verso la classe **Team**),
- **Competizione** (verso la classe **Hackathon**),
- **Organizzazione** (verso la classe **Hackathon**).

Sono state progettate per consentire una gestione dello **storico degli eventi e delle interazioni tra le classi del sistema**. Nel contesto di una piattaforma per la gestione e la memorizzazione di molteplici hackathon, è stato ritenuto di notevole importanza mantenere uno storico che consenta di tracciare in modo completo e coerente il ciclo di vita di ogni oggetto. Tale approccio aggiunge alla piattaforma:

- una **tracciabilità temporale** delle attività
- la possibilità di effettuare **analisi retrospettive** e di generare report dettagliati.
- **scalabilità**: prepara il sistema a possibili **evoluzioni future**

Quindi, dal punto di vista pratico sarà possibile ottenere:

- per ogni Partecipante: a quali Team ha partecipato
- per ogni Team: a quali Hackathon ha partecipato
- per ogni Organizzatore: quali Hackathon ha organizzato

Tutto ciò appena esposto ha l'obiettivo di garantire **coerenza**, **completezza** e **scalabilità** alla progettazione del sistema.

Associazione Ternaria (M:N:P)

L'associazione di **Selezione** è espressa tramite uno strumento di raro utilizzo ma di grande utilità: una **associazione n-aria**. Questa associazione è frutto di un'estensione di quella binaria tra **Organizzatore** e **Giudice**, che non presenterebbe modo di rintracciare l'**Hackathon** di riferimento per via dell'implementazione dello *storico degli eventi*. Così facendo introduciamo un legame univoco per ogni diverso **Hackathon**, permettendo che lo stesso **Organizzatore** inviti gli stessi **Giudici** in diverse competizioni.

2.3.2 Identificazione Attributi

In questa fase vengono individuate le **informazioni da conservare** per ciascuna classe, analizzando i dati necessari al corretto funzionamento del sistema.

- Durante il processo, alcuni **nomi precedentemente scartati come classi** possono essere reinterpretati come **attributi**, se rappresentano semplici proprietà piuttosto che entità autonome.
- Ogni **attributo** dovrebbe generalmente contenere un **solo valore**, al fine di mantenere la coerenza e la chiarezza del modello dei dati.
- È inoltre importante evitare la presenza di **attributi duplicati**, in quanto ridondanze di questo tipo possono complicare la gestione e l'evoluzione del sistema.
- Infine, qualora un sottoinsieme di attributi di una classe costituisca un **gruppo coerente**, è consigliabile creare una **classe distinta** per rappresentarli, favorendo una struttura più modulare e organizzata.

Attributi delle Classi

Di seguito gli attributi caratterizzanti rispettivamente ogni classe del modello del dominio del problema:

Utente

- Nome
- Cognome
- Username
- Password

Team

- Nome
- NumeroMembri

Documento

- Titolo
- Contenuto
- Commento

Hackathon

- Titolo
- Sede
- Durata
- DataInizio
- DataFine
- DataAperturaIscrizioni
- DataChiusuraIscrizioni
- MaxIscritti
- MaxDimTeam
- DescrizioneProblema

Attributi delle Classi Associative

Registrazione - Data

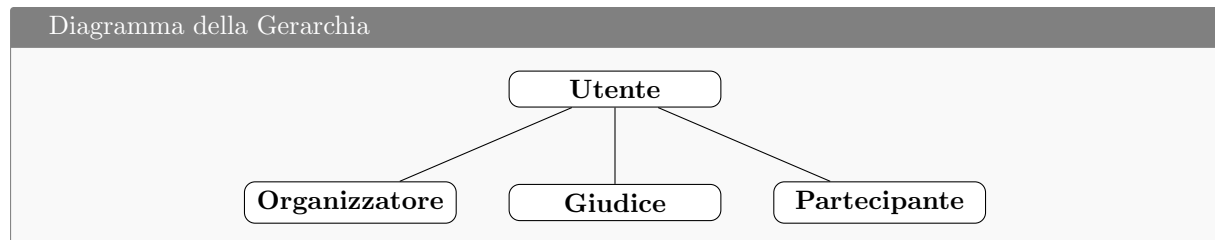
Votazione - Voto

2.4 Identificazione Generalizzazioni ed Interfacce

Le generalizzazioni ed interfacce individuate sono state individuate seguendo un processo **Top-Down**: a partire dalle classi più generali per poi specializzarle.

2.4.1 Generalizzazioni Individuate

È stata individuata la classe generale **Utente**, dalla quale derivano tre classi specializzazione: **Organizzatore**, **Giudice** e **Partecipante**. Tale gerarchia consente di modellare in maniera più accurata i diversi ruoli che gli utenti possono assumere all'interno di un **Hackathon**.



2.4.2 Vincoli della Generalizzazione

I vincoli che caratterizzano la gerarchia sono:

- **Vincolo di Overlapping**: un'istanza può appartenere a *più* di una sottoclasse.
- **Vincolo di Completezza Parziale**: un'istanza della superclasse non è obbligata a far parte di una sottoclasse.

Questo vincolo riguarda la piattaforma nella sua interezza ma, come specificato precedentemente, nel contesto di un singolo **Hackathon** assume la forma di un **vincolo di disgiunzione**.

Tali vincoli garantiscono il corretto funzionamento del sistema, in accordo con l'analisi dei requisiti:

- Un **Utente** "diventa" **Partecipante** se e solo se ($\Leftrightarrow$) completa la registrazione ad un **Hackathon**.
Gli utenti generici non assumono automaticamente questo status.
- I **Giudici** hanno un ruolo esclusivo: possono valutare i progetti ma non iscriversi come partecipanti.
- Gli **Organizzatori** hanno accesso solo alle funzioni di gestione e non possono partecipare direttamente alla competizione.

2.4.3 Interfacce Individuate

Le interfacce non sono state ritenute uno strumento utile per modellare alcun oggetto del mondo reale appartenente al modello del dominio del problema.

2.5 Modello di Dominio del Problema

Di seguito il modello del dominio del problema espresso tramite **Class Diagram** seguendo gli standard UML

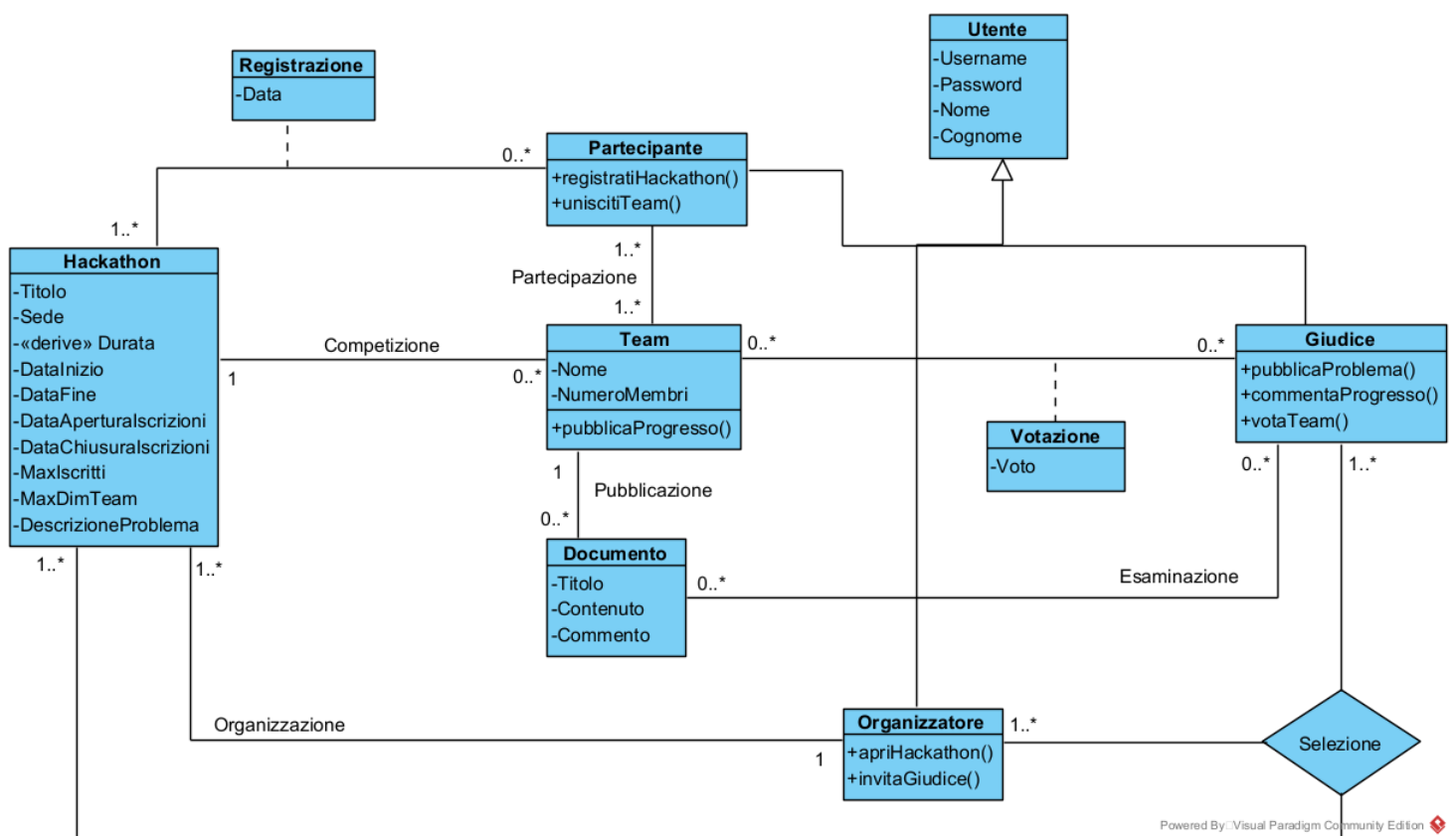


Figura 1: Modello del Dominio del Problema - Diagramma delle Classi UML

3 Dizionari

3.1 Dizionario delle Classi

Classe	Descrizione	Attributi	Responsabilità
Utente	Classe base per tutti gli utenti del sistema	- Nome - Cognome - Username - Password	- Registrazione alla piattaforma
Partecipante	Utente registrato come partecipante		- Iscrizione ad un Hackathon - Creazione di un Team - Entra a far parte di un Team
Giudice	Utente abilitato alla valutazione		- Pubblicazione del problema - Aggiunta di un commento al Progresso di un Team - Aggiunta di un Voto ad un Team
Organizzatore	Utente che gestisce gli eventi		- Invitare i Giudici all'Hackathon - Aprire le Iscrizioni ad un Hackathon
Team	Gruppo di partecipanti	- Nome - NumeroMembri	- Pubblicazione dei Progressi di un Team
Documento	File prodotto durante l'hackathon	- Titolo - Contenuto - Commento	
Hackathon	Competizione di programmazione	- Titolo - Sede - Durata - DataInizio - DataFine - DataAperturaIscrizioni - DataChiusuraIscrizioni - MaxPartecipanti - DimMaxTeam - DescrizioneProblema	

Tabella 1: Descrizione delle classi del Modello di Dominio del problema

3.2 Dizionario delle Associazioni

Relazioni	Tipologia	Descrizione
Competizione	Uno-a-Molti	- Un Team compete in un 1 singolo Hackathon - In 1 Hackathon competono nessuno o più Team
Pubblicazione	Uno-a-Molti	- Un Team pubblica nessuno o più Documenti - Un Documento è pubblicato da 1 ed un solo Team
Esaminazione	Molti-a-Molti	- Un Giudice esamina nessuno o più Documenti - Un Documento è esaminato da nessuno o più Giudici
Organizzazione	Uno-a-Molti	- Un Organizzatore organizza 1 o più Hackathon - Un Hackathon è organizzato da 1 Organizzatore
Selezione	Molti-a-Molti-a-Molti	- Un Organizzatore seleziona 2 o più Giudici nel contesto di 1 o più Hackathon - Un Giudice è selezionato da 1 o più Organizzatori nel contesto di 1 o più Hackathon - In un Hackathon sono selezionati 2 o più Giudici da 1 Organizzatore
Partecipazione	Molti-a-Molti	- Un Partecipante partecipa ad 1 o più Team - Un Team possiede 1 o più Partecipanti
Registrazione	Molti-a-Molti	- Data: attributo di relazione per memorizzare la data di Registrazione all'Hackathon - Un Partecipante si registra a 1 o più Hackathon - Ad un Hackathon si registrano nessuno o più Partecipanti
Votazione	Molti-a-Molti	- Voto: attributo di relazione che individua il valore numerico della votazione - Un Team è valutato da nessuno o più Giudici - Un Giudice valuta nessuno o più Team

Tabella 2: Descrizione delle associazioni tra le classi individuate

3.3 Dizionario Classi Associative

Classe Associativa	Attributi	Descrizione
Registrazione	Data	Azione con cui un utente partecipa ad un hackathon
Votazione	Voto	Valutazione data da un giudice ad un team

Tabella 3: Classi associative e relativi attributi di associazione

Introduzione all'Implementazione del Modello di Dominio del Problema

Per giungere all'implementazione del modello di dominio formato dagli tutti gli elementi appena esposti, e quindi alla **definizione del modello di dominio della soluzione**, è stato ritenuto necessario stabilire delle regole assiomatiche. L'introduzione di assiomi rappresenta una scelta metodologica deliberata, finalizzata a stabilire i principi fondamentali che governeranno l'intero sistema informativo **prima** dell'avvio della fase di **implementazione del modello di dominio del problema**. Nello *scope* della presente piattaforma, la definizione di assiomi evita **un'eccessiva granularità nell'implementazione**. L'approccio assiomatico permette di identificare quali **vincoli** richiedano un'**implementazione procedurale complessa** e quali possano essere gestiti attraverso **meccanismi più semplici** o **delegati** ad altri livelli dell'architettura applicativa.

Assiomi per l'implementazione del Modello di Dominio del Problema

Le **definizioni** degli elementi di seguito menzionati si trovano alla sezione [Formalizzazione dei Vincoli](#).

(1) Immutabilità degli Attributi di Hackathon

Sia H un hackathon e sia S lo stato di creazione della competizione. Una volta raggiunto lo stato S , tutti gli attributi configurazionali di H divengono **immutabili** per garantire la consistenza referenziale dell'intera base di dati.

Formalmente:

$$\boxed{\forall a \in \text{Attributi}(H), t \geq t_S : \text{Immutabile}(a, t)}$$

dove t_S è il tempo di raggiungimento dello stato S e $\text{Immutabile}(a, t)$ indica che l'attributo a non può essere modificato al tempo t .

Tale principio assicura l'integrità dei dati correlati e previene incongruenze sistemiche derivanti da modifiche post-creazione.

(2) Completezza della Votazione

La procedura di votazione da parte di un giudice è **corretta** e ha **significato** se e solo se ogni Team ha pubblicato tutti i propri documenti prima della fine dell'Hackathon.

Formalmente:

Per ogni giudice $g \in G$, la procedura di votazione è **corretta** se e solo se:

$$\text{Votazione}(g) \Leftrightarrow \forall d \in D, \text{Pubblicato}(d, t_f)$$

dove $\text{Pubblicato}(d, t_f)$ indica che il documento d è stato pubblicato entro il tempo di fine t_f dell'hackathon H .

Questo principio stabilisce una convenzione contrattuale tra team e giudici, secondo cui il processo valutativo può essere avviato unicamente a seguito della pubblicazione integrale della documentazione richiesta, e deve essere completato entro i termini dell'hackathon.

(3) Transizioni Temporali degli Stati di Hackathon da parte dell'Organizzatore

Sia H un hackathon organizzato da $o \in O$, con i seguenti stati:

- S_C : stato di **creazione**;
- S_I : stato di **inizio**;
- S_F : stato di **fine**.

L'organizzatore deve garantire che l'hackathon:

1. ($S_C \rightarrow S_i$) transiti nello stato S_i entro la data pianificata in S_C (**dataInizio**);
2. termini ($S_i \rightarrow S_f$) esattamente nella data pianificata (**dataFine**);
3. non possa tornare a stati precedenti una volta raggiunto S_I o S_F .

Formalmente:

- Quando H è nello stato di creazione (S_C), l'organizzatore o può permetterne la transizione nello stato di inizio (S_I) se e solo se **dataCorrente** = (**dataInizio**) e **non prima**.
- Quando H è nello stato di inizio (S_I), l'organizzatore o può permetterne la transizione nello stato di fine (S_F) se e solo se **dataCorrente** = (**dataFine**) e **non dopo**.
- Nessun ritorno agli stati precedenti è permesso: una volta iniziato, H non può tornare a S_C , e una volta terminato, non può tornare a S_I .

In altre parole, le transizioni ammesse sono:

$$S_C \xrightarrow{\text{dataInizio}} S_I \xrightarrow{\text{dataFine}} S_F$$

e ogni transizione deve avvenire esattamente nelle date pianificate.

4 Packages

Implementazione del Modello di Dominio del Problema

Per la realizzazione del modello del dominio della soluzione, come richiesto dalla traccia di progetto, è stata effettuata l'implementazione delle classi e delle associazioni del modello di dominio del problema tramite il linguaggio di programmazione **Java**.

Quest'ultimo offre la possibilità di poter suddividere un progetto in componenti definiti come **packages**.

Definizione: Un **package** in Java è un meccanismo per organizzare classi, interfacce e altri elementi correlati in gruppi logici.

In questa sezione quindi, vengono così descritti i packages individuati utilizzando i principi del pattern architetturale **Boundary-Control-Entity + Data Access Object (BCE + DAO)** il quale prevede la definizione di packages che rispettivamente descrivono ogni componente del pattern:

- Package **GUI** (Boundary)
- Package **Controller** (Control)
- Package **Model** (Entity)
- Package **DAO**

Pattern Architetturale

Boundary-Control-Entity (BCE)

Il pattern **Boundary-Control-Entity (BCE)** è stato adottato per organizzare il sistema in tre tipologie di componenti con responsabilità ben definite:

- **Boundary:** Gestisce l'interazione con attori esterni (interfacce utente, sistemi esterni, database)
- **Control:** Coordina le operazioni applicative e implementa la logica di controllo
- **Entity:** Rappresenta i concetti persistenti del dominio e la logica di business fondamentale

Questa architettura garantisce una chiara tracciabilità dei requisiti funzionali e facilita l'evoluzione del sistema mantenendo un basso accoppiamento tra i componenti.

Data Access Object (DAO)

Il pattern **Data Access Object (DAO)** è invece adottato per rappresentare correttamente le interazioni con un sistema di **persistenza dei dati**. L'architettura DAO prevede l'introduzione di **interfacce** per le funzioni di memorizzazione e recupero dati richieste dal sistema, accoppiate con una relativa classe **concreta** per l'effettiva implementazione di essa. In particolare:

- Il package **DAO** si occupa della definizione delle interfacce
- Il package **postgresImplementationDAO** si occupa dell'implementazione di esse utilizzando un *RDBMS* designato (PostgreSQL).

Flusso di Interazione e Relazioni BCE + DAO

L'architettura BCE prevede un flusso di interazione ben definito tra i componenti:

1. L'utente interagisce con i **Boundary** del package **GUI**
2. I **Boundary** notificano i **Control** del package **Controller**
3. I **Control** elaborano la richiesta, coordinano le **Entity** del package **Model** e, se necessario, rendono persistenti i cambiamenti grazie al package **DAO** per la memorizzazione
4. Le **Entity** applicano la logica di business
5. I **Control** aggiornano i **Boundary** della **GUI** con i risultati

4.1 Package GUI

Responsabilità: Gestisce l'interfaccia grafica utente

Componenti BCE: Boundary

Contenuto: Classi per la visualizzazione e l'interazione con l'utente

- Implementa i Boundary dell'interfaccia grafica (finestre, form, controlli)
- Si occupa del rendering dei dati e della raccolta dell'input utente
- Applica le logiche di presentazione senza contenere regole di business
- Comunica con i componenti Control per trasmettere le azioni dell'utente
- Fornisce feedback visivo all'utente finale

Package GUI Diagram

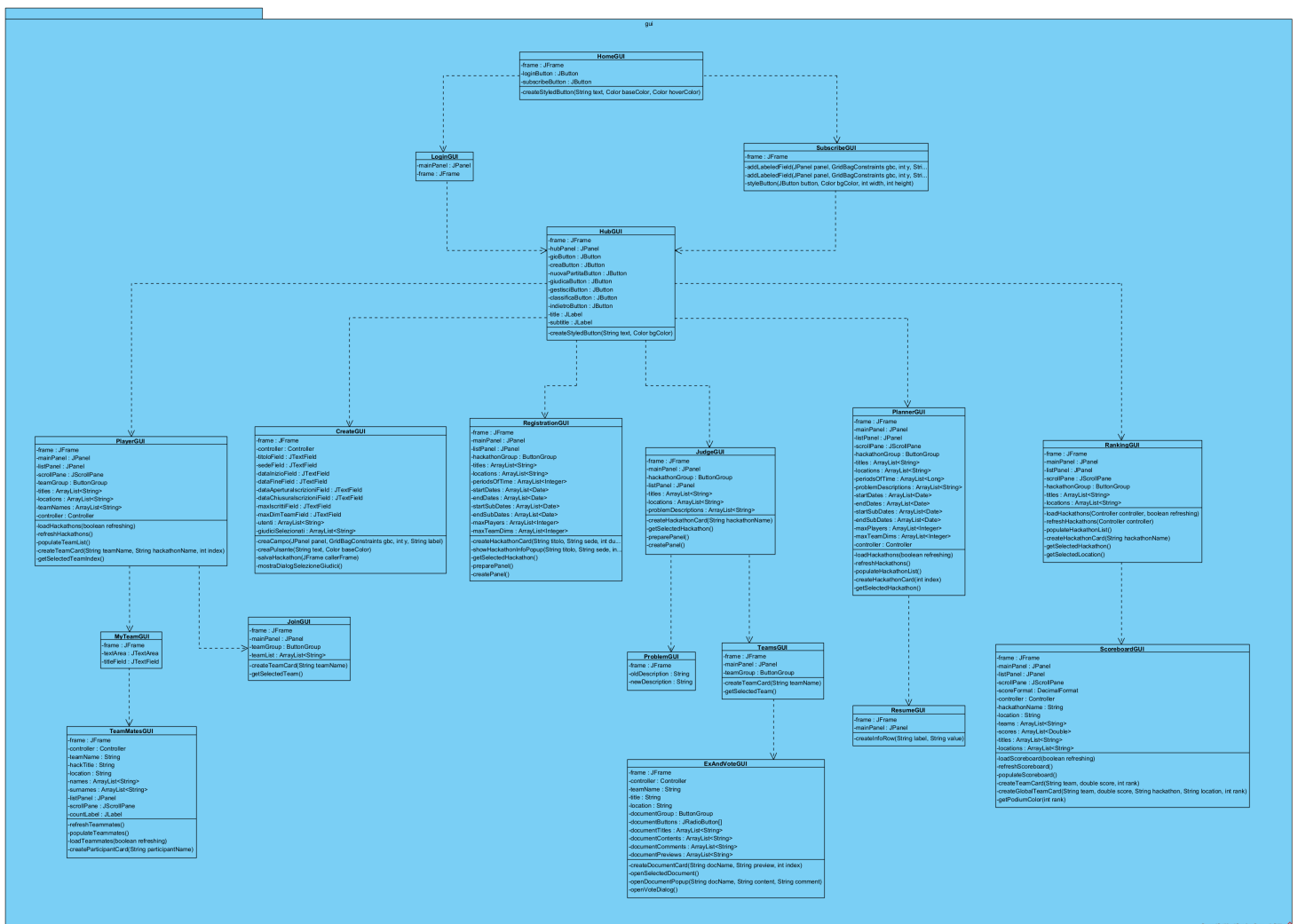


Figura 2: Package GUI Diagram

4.2 Package Controller

Responsabilità: Coordina il flusso dell'applicazione e implementa la logica di controllo

Componenti BCE: Control

Contenuto: Classi che gestiscono la logica di controllo e coordinamento

- Implementa i componenti Control che incapsulano casi d'uso specifici
- Riceve le richieste dall'utente attraverso i Boundary della GUI
- Orchestra le operazioni applicative coinvolgendo le Entity del Model
- Gestisce la logica transazionale e il flusso di lavoro
- Coordina multiple Entity per realizzare operazioni complesse

Package Controller Diagram

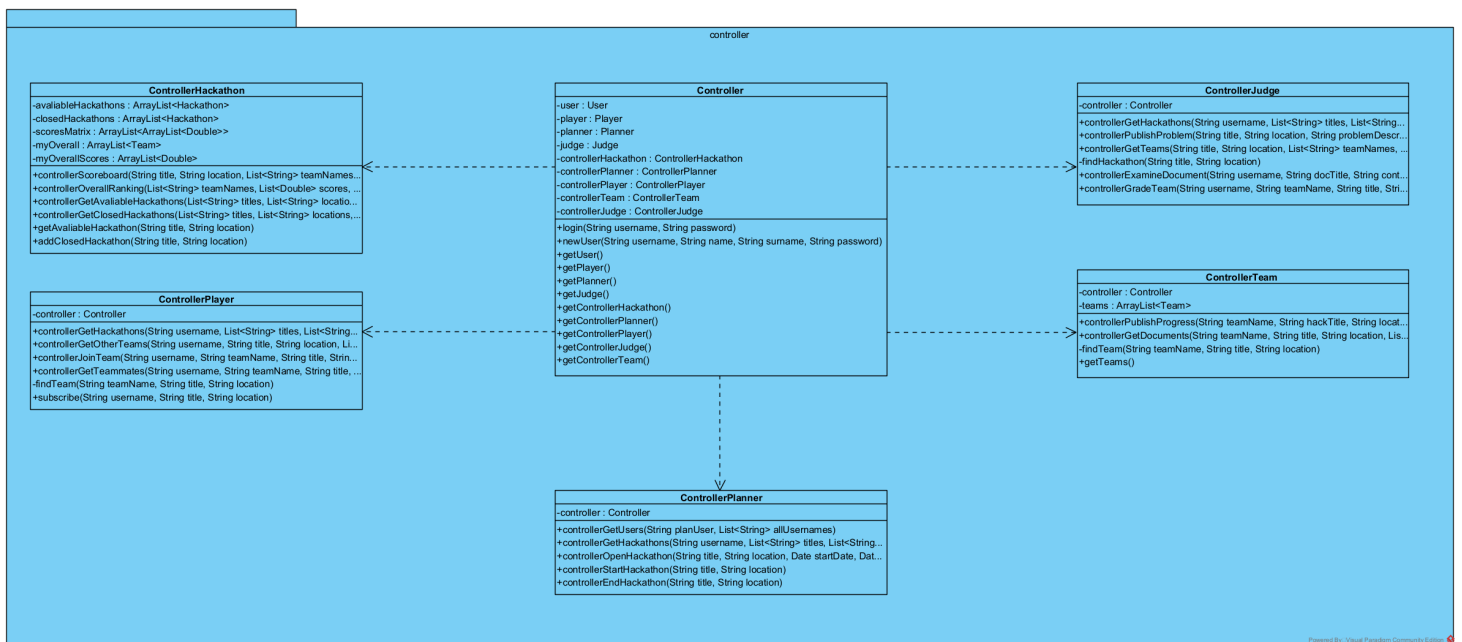


Figura 3: Package Controller Diagram

4.3 Package Model

Responsabilità: Implementa le entità del dominio e la logica di business fondamentale

Componenti BCE: Entity

Contenuto: Classi che rappresentano il modello di dominio del problema

- Contiene le classi Entity che mappano le entità del dominio analizzato
- Implementa le regole di business invarianti e le validazioni dei dati
- Definisce le associazioni tra le classi come specificato nel modello di dominio
- È completamente indipendente dall'interfaccia utente e dai meccanismi di persistenza
- Rappresenta lo stato locale dell'applicazione

Package Model Diagram

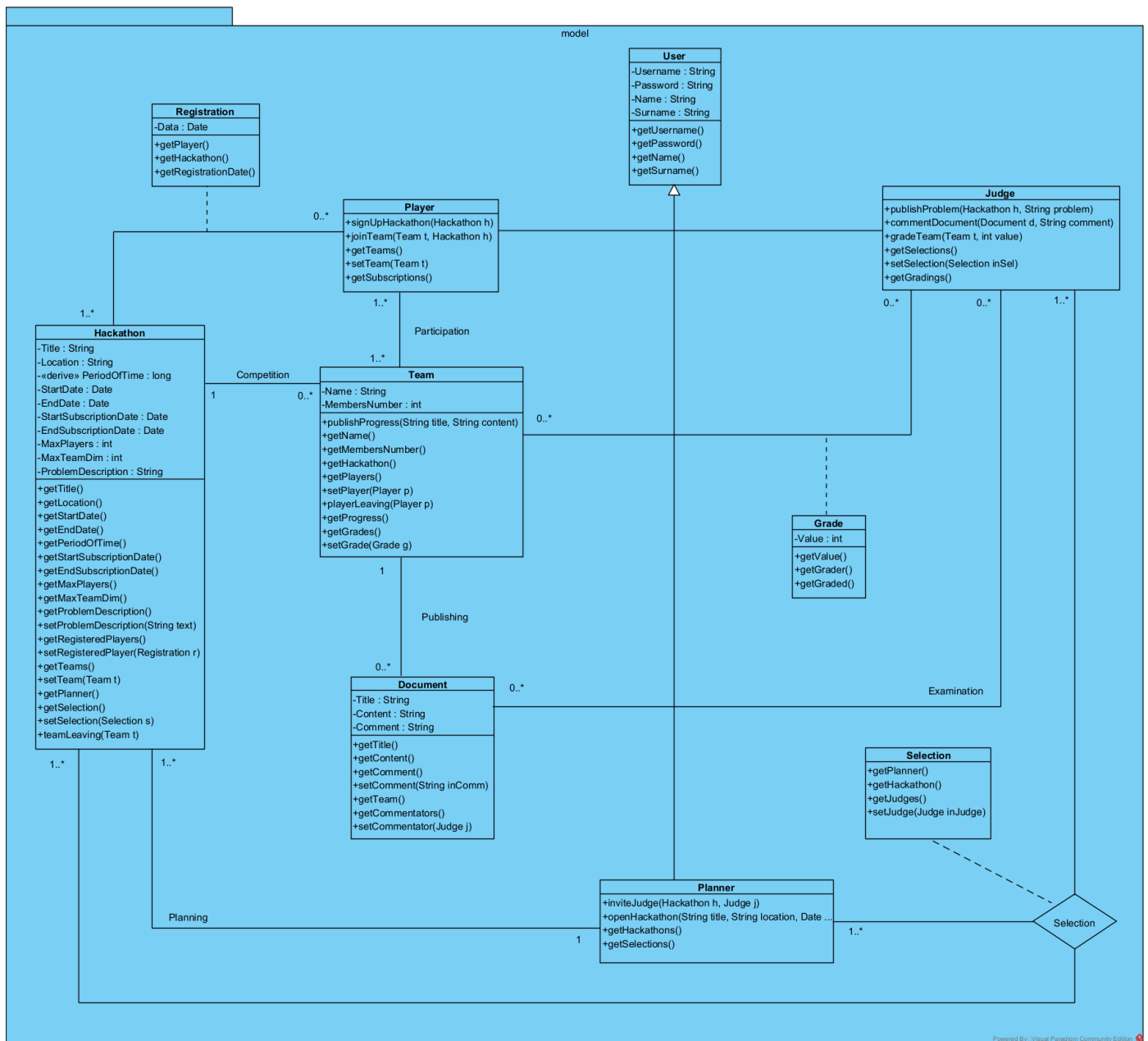


Figura 4: Package Model Diagram

4.4 Package DAO

Responsabilità: Gestisce l'accesso ai dati e la persistenza

Contenuto: Classi per l'interazione con il sistema di persistenza

- Definisce e implementa le interfacce che comunicano con il database
- Astrae le operazioni di persistenza (CRUD) per le Entity del Model
- Incapsula la logica di accesso al database e i dettagli tecnologici
- Isola il resto dell'applicazione dai meccanismi specifici di storage
- Gestisce connessioni, transazioni e mapping oggetto-relazionale

Package DAO Diagram

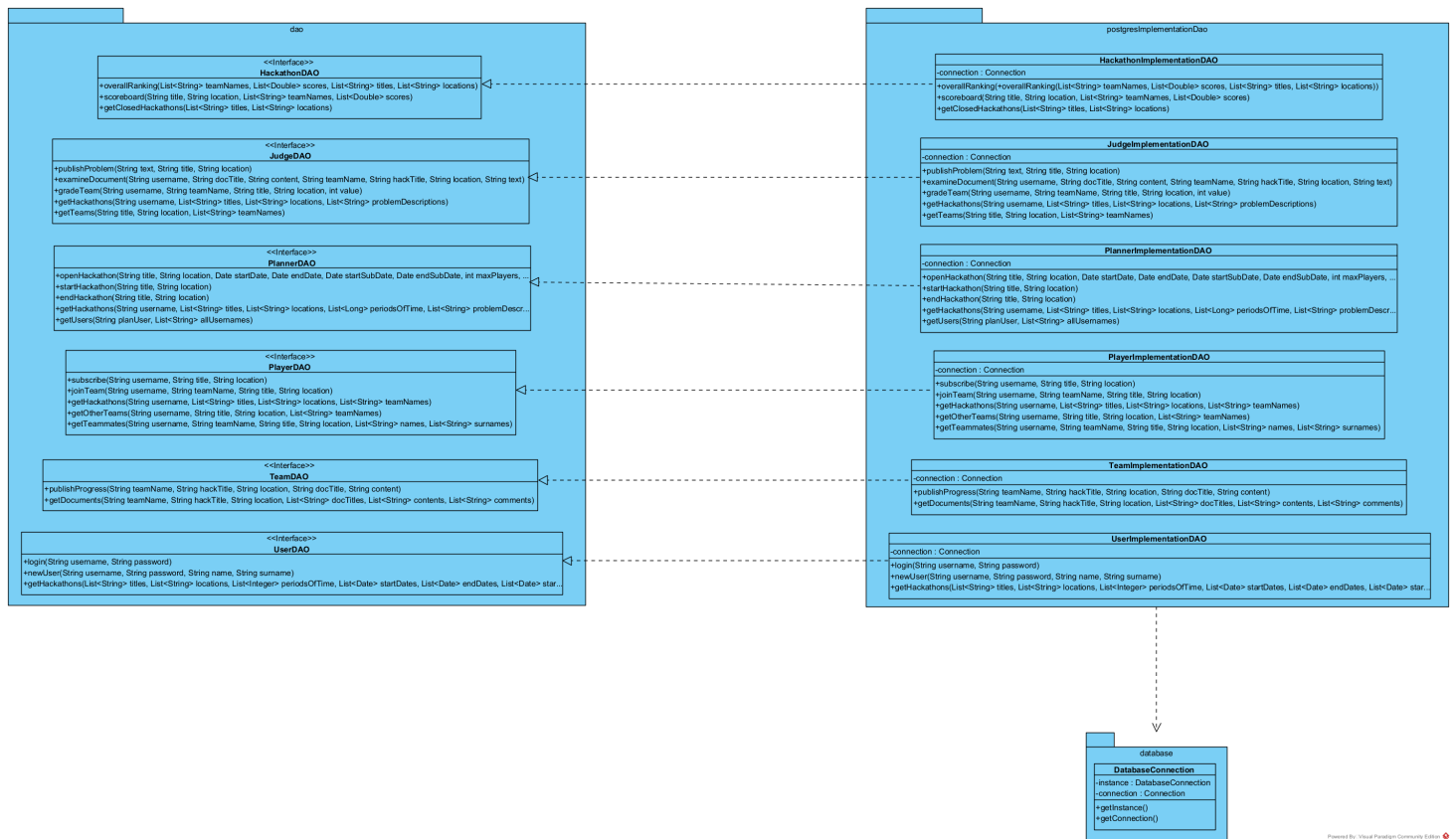


Figura 5: Package DAO Diagram

5 Dominio della Soluzione

Definizione: Il **Dominio della Soluzione** (Solution Domain) rappresenta l'implementazione tecnica e concettuale del sistema software, comprendendo tutti gli elementi architetturali, le tecnologie, i design pattern e le scelte progettuali specifiche che realizzano la soluzione al problema delineato nel Dominio del Problema.

5.1 Solution Diagram

Di seguito il **Modello del Dominio della Soluzione** espresso tramite *Package Diagram* seguendo i principi dello standard UML.

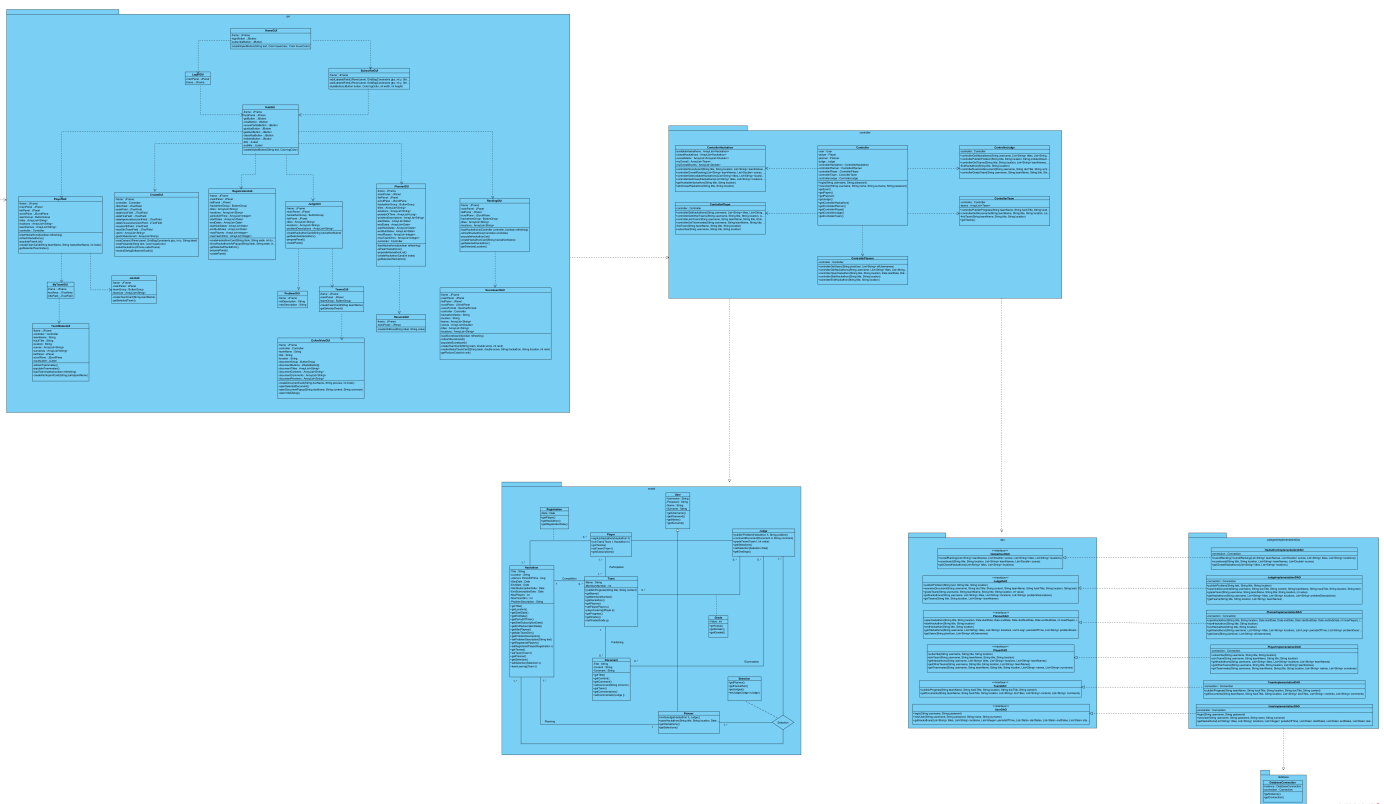


Figura 6: Solution Diagram

6 Sequence Diagrams

Definizione: Un **Sequence Diagram** è un diagramma UML che mostra le interazioni tra oggetti o componenti in un sistema in funzione del tempo. Illustra come e in quale sequenza gli oggetti si scambiano messaggi per realizzare una specifica funzionalità.

6.1 Creazione di un Hackathon

Di seguito è illustrato il *Sequence Diagram* che descrive il processo operativo relativo alla creazione di un nuovo hackathon nella piattaforma da parte di un utente che rivestirà il ruolo di Organizzatore.

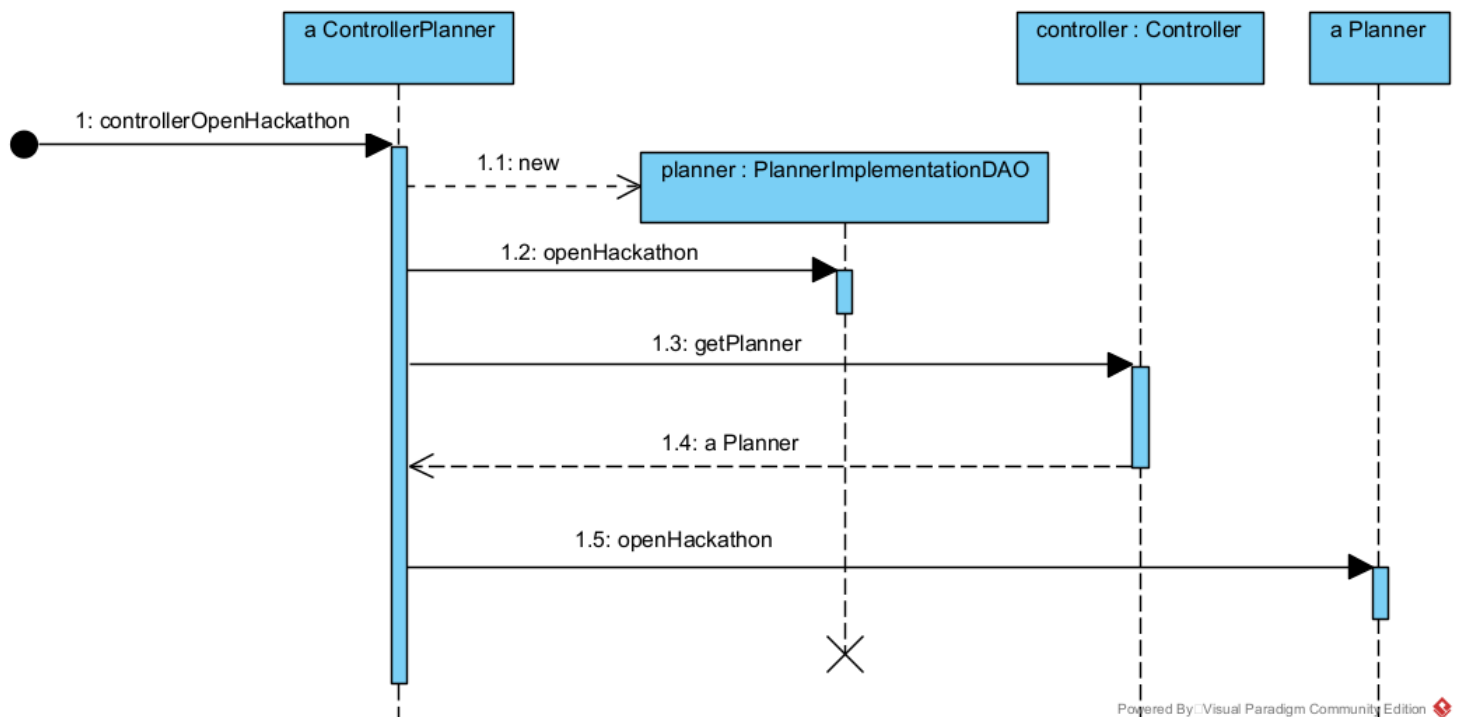


Figura 7: Sequence Diagram rappresentante il processo di creazione di un hackathon

6.2 Pubblicazione di un Progresso

In figura è rappresentanto il *Sequence Diagram* che descrive il processo operativo relativo alla pubblicazione di un Progresso (Documento) da parte di un Team partecipante ad un Hackathon.

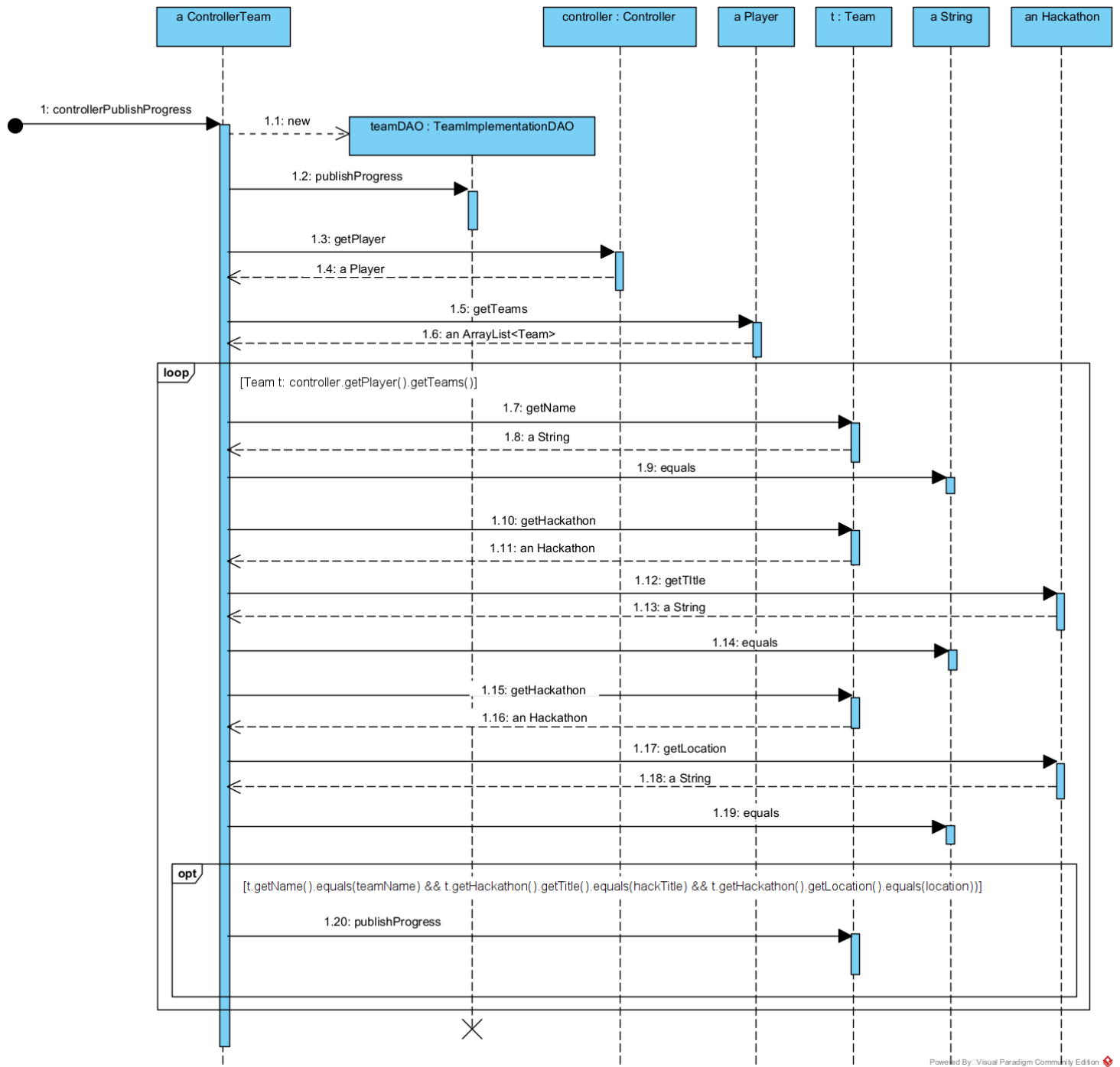


Figura 8: Sequence Diagram rappresentante il processo di pubblicazione di un progresso da parte di un Team partecipante ad un Hackathon

7 Manuale per l'uso

La presente sezione ha l'obiettivo di **descrivere dettagliatamente le azioni da seguire per un corretto utilizzo della piattaforma HackathON**:

in particolare viene illustrato ogni componente della UI indicando il modo tale da utilizzare le funzionalità e visionarne l'effetto. Queste ultime sono corrispondenti secondo una relazione 1:1 ai *Requisiti Funzionali* precedentemente indicati.

Convenzioni Notazionali			
1.n ($n \in \mathbb{N}^+$)	Bottoni dell'interfaccia	→ XGUI	reindirizzamento verso una GUI <i>non ancora visitata</i>
2.m ($m \in \mathbb{N}^+$)	Campi di inserimento	YGUI ←	reindirizzamento verso una GUI <i>già visitata</i>
3.k ($k \in \mathbb{N}^+$)	Campi testuali	ZGUI	classe GUI corrente

HackathON! — Home Page



Figura 9: HomeGUI

La prima componente dell'interfaccia utente, *HomeGUI*, presenta un messaggio di benvenuto e i seguenti pulsanti:

#	Elemento	Classe GUI	Descrizione
1.1	Login	→ LoginGUI	Bottone per accedere alla GUI di login
1.2	Registrati	→ RegistrationGUI	Bottone per accedere alla GUI per la registrazione
1.3	Chiudi	∅ ←	Terminazione del programma

HackathON! — Login

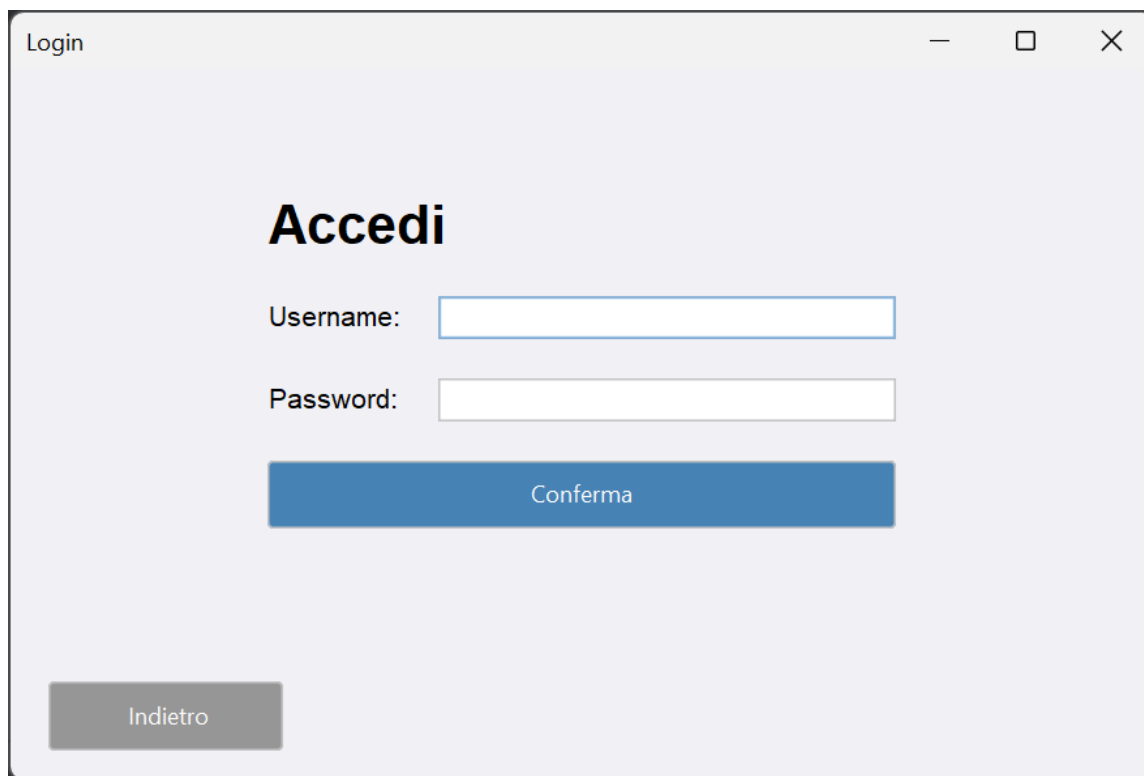


Figura 10: Interfaccia grafica di login

La seconda componente dell'interfaccia utente, *LoginGUI*, presenta i seguenti elementi:

#	Elemento	Classe GUI	Descrizione
1.4	Conferma	→ HubGUI	Conferma delle credenziali per il login nella piattaforma
1.5	Indietro	→ HomeGUI	Ridireziona alla componente GUI precedente
2.1	Username	LoginGUI	Campo per l'inserimento dell'username
2.2	Password	LoginGUI	Campo per l'inserimento della password

HackathON! — Registrazione

Registrazione Utente - HackathON

Iscriviti alla piattaforma

Nome:

Cognome:

Username:

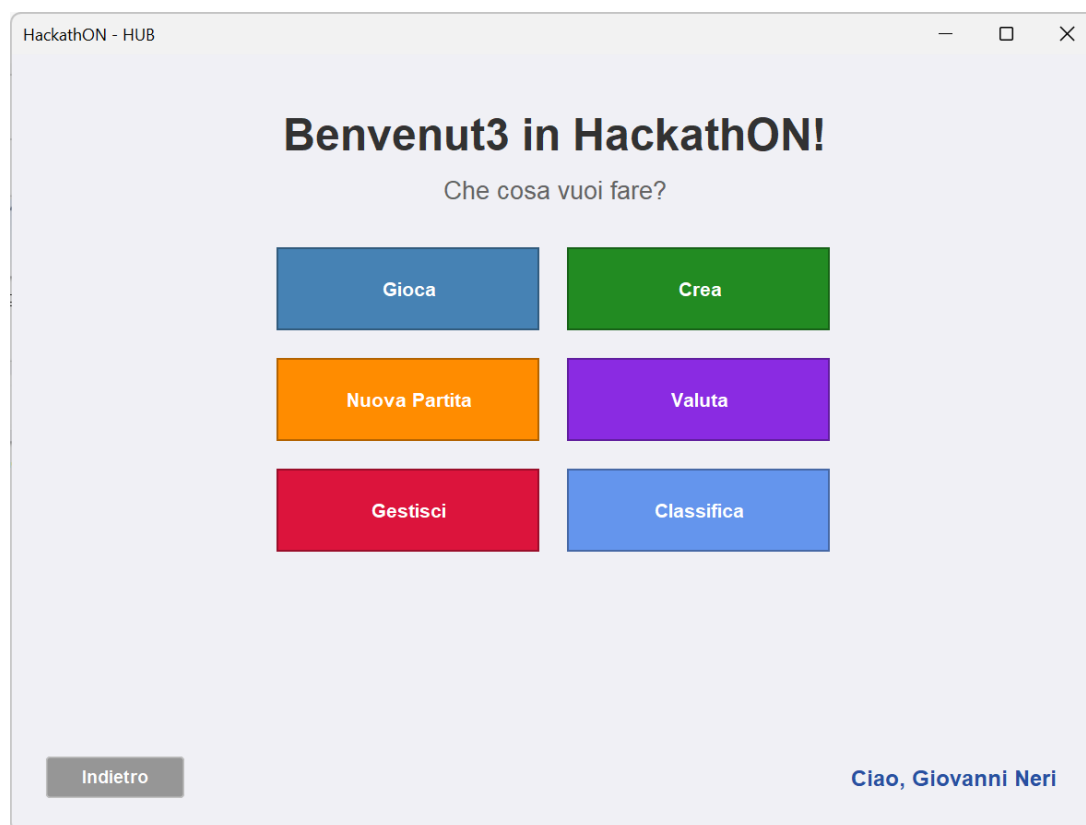
Password:

Conferma

Indietro

#	Elemento	Classe GUI	Descrizione
1.6	Conferma	→ HubGUI	Conferma delle credenziali per la registrazione nella piattaforma
1.7	Indietro	→ HomeGUI	Ridireziona alla componente GUI precedente (HomeGUI)
2.3	Nome	SubscribeGUI	Campo per l'inserimento del nome
2.4	Cognome	SubscribeGUI	Campo per l'inserimento del cognome
2.5	Username	SubscribeGUI	Campo per l'inserimento dell'username
2.6	Password	SubscribeGUI	Campo per l'inserimento della password

HackathON! — HUB



#	Elemento	Classe GUI	Descrizione
1.8	Gioca	→ PlayerGUI	Visualizzazione dell'elenco di Team in cui l'utente loggato risulta Partecipante
1.9	Crea	→ CreateGUI	Visualizzazione GUI per la creazione di un Hackathon; l'utente loggato ne diventa così l'Organizzatore
1.10	Nuova Partita	→ RegistrationGUI	Visualizzazione dell'elenco di Hackathon a cui è possibile registrarsi
1.12	Valuta	→ JudgeGUI	Visualizzazione degli Hackathon in cui l'utente che ha effettuato il login è giudice
1.13	Gestisci	→ PlannerGUI	Visualizzazione degli Hackathon in cui l'utente che ha effettuato il login è organizzatore
1.14	Classifica	→ ScoreboardGUI	Visualizzazione dell'elenco di Hackathon terminati
1.15	Indietro	LoginGUI ← SubscribeGUI ←	Ridireziona alla componente GUI precedente

Nota 1: Gestione dei Ruoli Multipli

Un utente autenticato sulla piattaforma **HackathonON!** può ricoprire simultaneamente i seguenti ruoli:

Partecipante
Membro di team

Organizzatore
Creatore di eventi

Giudice
Valutatore di progetti

Vincolo: La sovrapposizione di ruoli è consentita esclusivamente in contesti di hackathon distinti. Un medesimo utente non può assumere ruoli multipli all'interno dello stesso evento.

Nota operativa: L'interfaccia HUB consente l'accesso a tutte le funzionalità associate ai diversi ruoli ricoperti dall'utente in **diversi hackathon**.

Nota 2.

Per gli utenti che hanno appena completato la registrazione, le funzionalità descritte nella Nota 1 non sono immediatamente disponibili, in quanto non è ancora stata stabilita alcuna affiliazione con eventi o team.

Nota implementativa: Il sistema verifica la presenza di **almeno un'entità associata** (team, hackathon organizzati, hackathon da valutare) prima di consentire l'accesso alle rispettive sezioni dell'interfaccia.

Casistiche e gestione degli errori:

Assenza del ruolo di Partecipante

- Viene visualizzato un messaggio di errore indicante l'assenza di affiliazioni a team attivi.

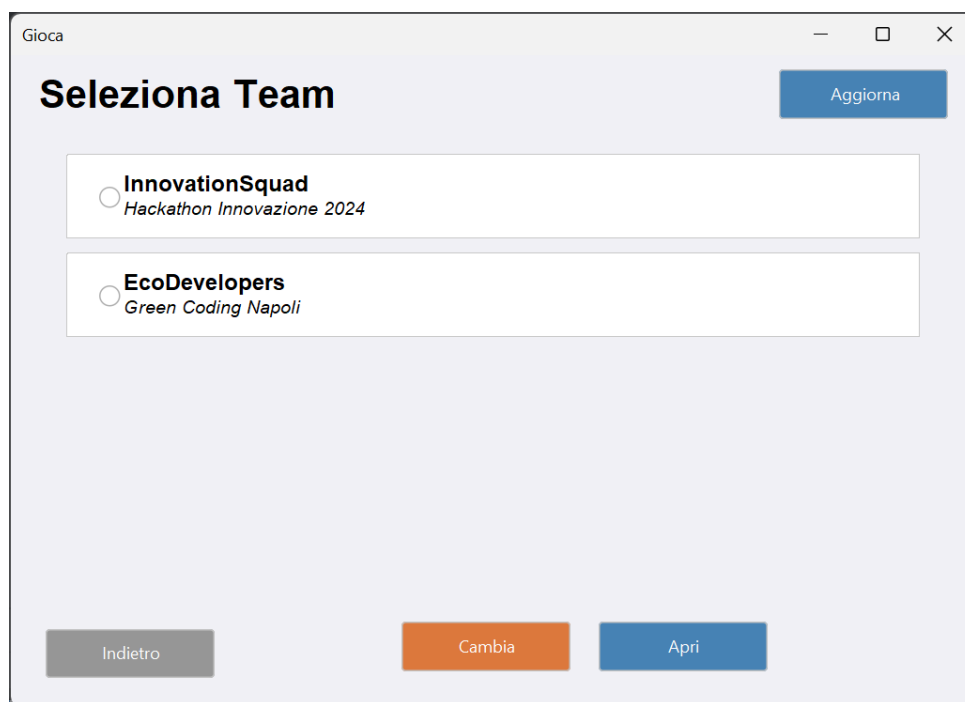
Assenza del ruolo di Organizzatore

- Viene visualizzato un messaggio di errore indicante l'assenza di hackathon creati dall'utente.

Assenza del ruolo di Giudice

- Viene visualizzato un messaggio di errore indicante l'assenza di valutazioni effettuate dall'utente in qualità di giudice.

HackathON! — Gioca



#	Elemento	Classe GUI	Descrizione
3.1	Team Card	PlayerGUI	Card con Radio Button di selezione di un Team di cui il giocatore fa parte
1.12	Indietro	HubGUI ←	Pulsante per tornare alla schermata precedente
1.13	Cambia	→ JoinGUI	Pulsante per modificare il proprio team
1.14	Apri	→ MyTeamGUI	Pulsante per procedere con la selezione di un team per la pubblicazione di un documento / visualizzazione dei componenti del Team

Nota

La presente componente GUI viene **visualizzata solo** nel momento in cui l'utente loggato risulti **partecipante ad almeno 1 Team**.

Hackathon! — Crea + Seleziona

Crea un Hackathon

Titolo:

Sede:

Durata:

Data Inizio (YYYY-MM-DD):

Data Fine (YYYY-MM-DD):

Apertura Iscrizioni (YYYY-MM-DD):

Chiusura Iscrizioni (YYYY-MM-DD):

Max Iscritti:

Max Dimensione Team:

Giudici:

Seleziona Giudici

Seleziona almeno 2 giudici:

☐ luca.bianchi

☐ anna.verdi

☐ giovanni.neri

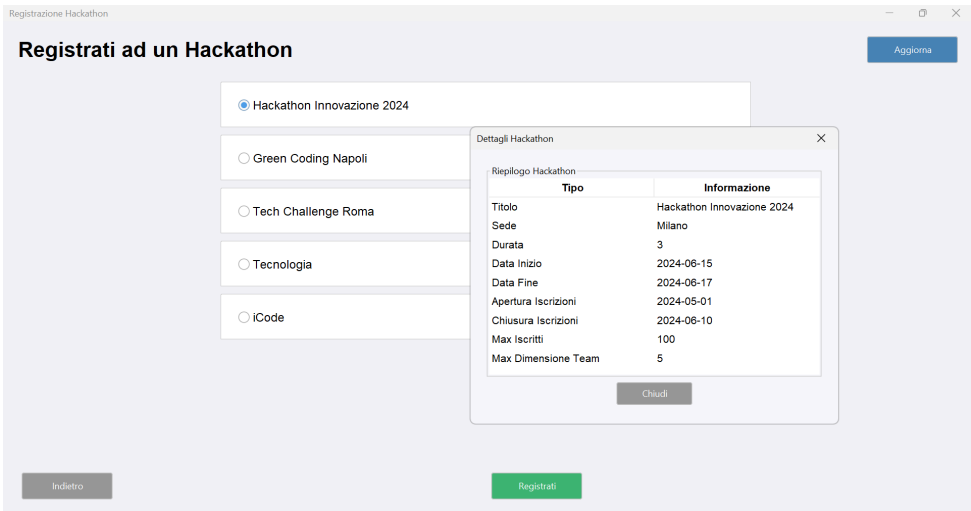
☐ sara.russo

☐ marco.ferrari

☐ elena.conti

#	Elemento	Classe GUI	Descrizione
2.7	Campi Testuali	CreateGUI	Gruppo di caselle di testo per l'inserimento dei dati dell'hackathon (Titolo, Sede, Durata, Date, Max Iscritti, Max Team, Giudici)
1.15	Seleziona	CreateGUI	Pulsante per selezionare gli utenti che diventeranno giudici dell'hackathon creato
1.16	Indietro	HubGUI ←	Pulsante per tornare alla schermata precedente
1.17	Salva	CreateGUI	Pulsante per salvare le informazioni dell'hackathon creato

HackathON! — Nuova Partita

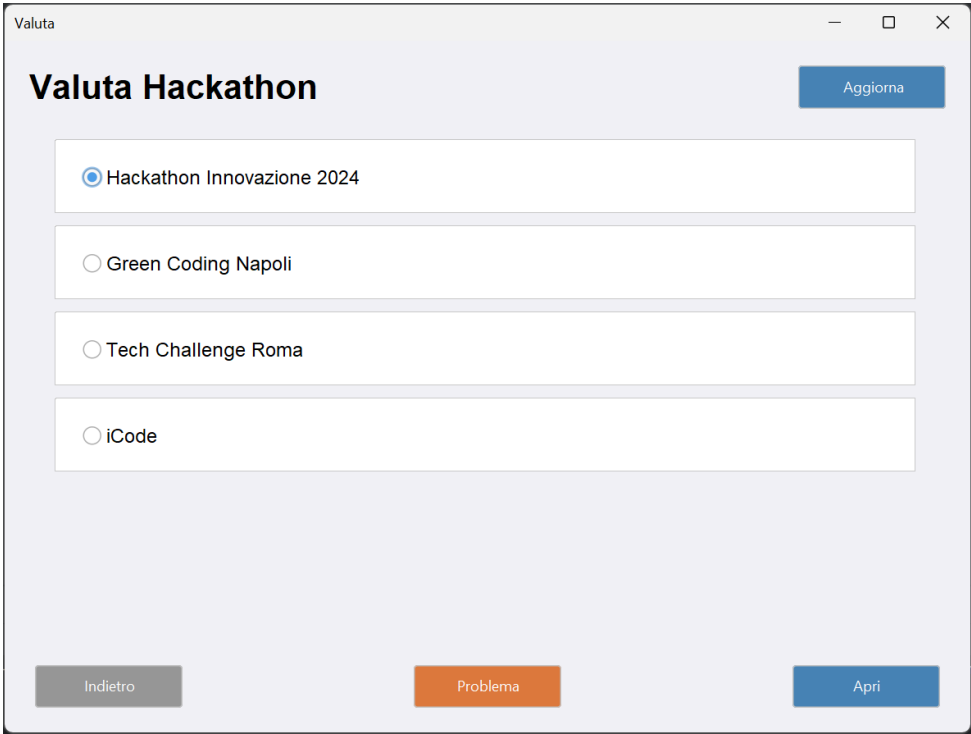


#	RegistrationGUI	Classe GUI	Descrizione
3.3	Hackathon Card	RegistrationGUI	Pulsante di selezione per registrarsi all'hackathon
1.18	Indietro	HubGUI ←	Pulsante per tornare alla schermata precedente
1.19	Registri	RegistrationGUI	Pulsante per confermare la registrazione all'hackathon selezionato e diventarne quindi partecipante

HackathON! — Valuta

Nota

La presente componente GUI, e di conseguenza tutte le interfacce la cui apertura avviene solo dopo avervi effettuato l'accesso (Valuta/X), vengono visualizzate esclusivamente nel momento in cui l'utente loggato risulti **giudice in almeno un Hackathon**.



#	Elemento	Classe GUI	Descrizione
3.4	Hackathon Card	JudgeGUI	card per la selezione per valutare l'hackathon
1.20	Indietro	HubGUI ←	Pulsante per tornare alla schermata precedente
1.21	Problema	→ ProblemGUI	Pulsante per la pubblicazione da parte del giudice del problema relativo all'Hackathon selezionato
1.22	Apri	→ TeamsGUI	Pulsante per aprire la schermata di valutazione dell'hackathon selezionato (funzionalità descritte successivamente)

HackathON! — Valuta/Problema

Problema - Hackathon Innovazione 2024

Problema Hackathon

Problema: Hackathon Innovazione 2024

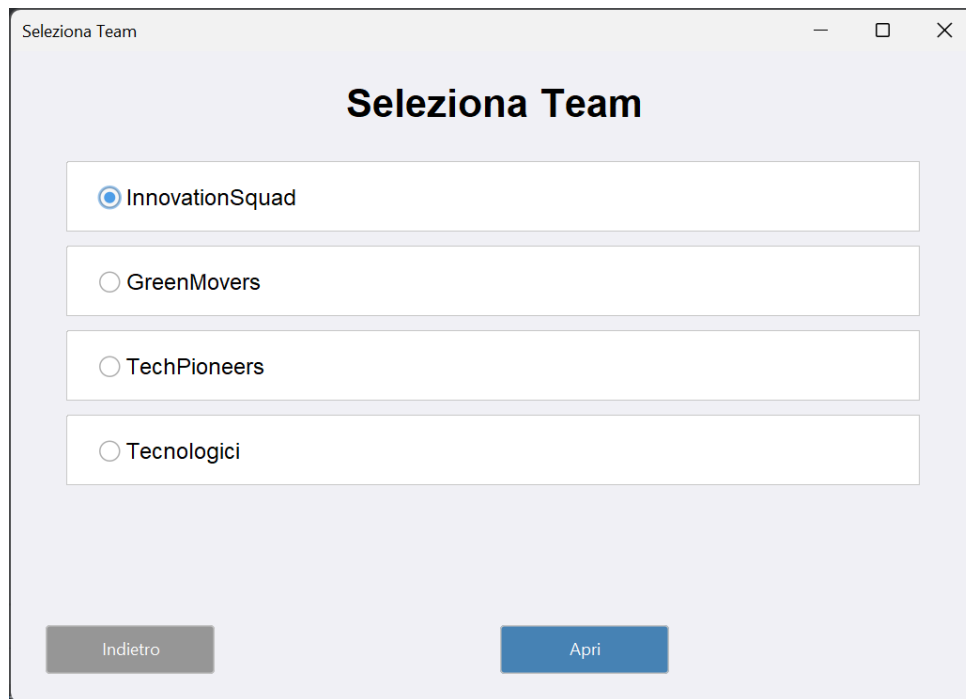
Sviluppare soluzioni innovative per la mobilita' sostenibile in ambito urbano

Indietro

Pubblica

#	Elemento	Classe GUI	Descrizione
2.8	Box Problema	ProblemGUI	Box in cui il giudice scrive la descrizione del problema
1.23	Indietro	JudgeGUI ←	Pulsante per tornare alla schermata precedente

HackathON! — Valuta/Apri



#	Elemento	Classe GUI	Descrizione
1.24	Apri	→ ExAndVoteGUI	Pulsante tramite il quale è possibile ottenere per il team selezionato l'elenco di documenti da questo pubblicati
1.25	Indietro	JudgeGUI ←	Pulsante per tornare alla schermata precedente
3.5	Team Card	TeamsGUI	card con radiobutton per selezione dei team di cui visualizzare il documento pubblicato per la valutazione

HackathON! — Valuta/Apri + Valutazione

Esamina Team - InnovationSquad

Team: InnovationSquad

☒ **Design System v2**
Nuove specifiche tecniche

☐ **Progetto Mobilità 4.0**
Contenuto progetto mobilità sostenibile

Assegna Voto

Seleziona un voto da 0 a 10

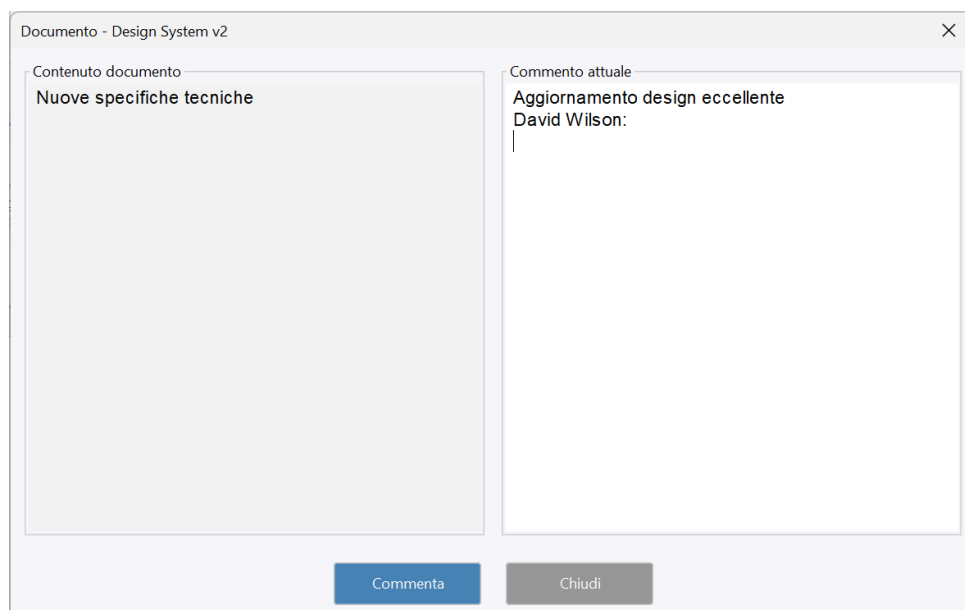
0

Conferma Annulla

Indietro Esamina Valuta

#	Elemento	Classe GUI	Descrizione
1.26	Conferma	ExAndVoteGUI	Pulsante tramite il quale è possibile confermare il voto assegnato al documento pubblicato dal team selezionato
2.9	Drop-Down Menu per Voto	ExAndVoteGUI	Drop down menu per la selezione di un voto $\in [0,10]$ per ogni documento pubblicato dal team selezionato
3.6	Document Card	ExAndVoteGUI	Card con radiobutton per selezionare il documento da Esaminare e/o valutare
1.27	Annulla	ExAndVoteGUI	Pulsante annullare la selezione del voto
1.28	Esamina	ExAndVoteGUI	pulsante per la visualizzazione del contenuto del documento selezionato
1.29	Valuta	ExAndVoteGUI	pulsante per la visualizzazione del drop-down menu per la selezione del voto da assegnare al documento

HackathON! — Valuta/Apri/Esamina



#	Elemento	Classe GUI	Descrizione
2.10	Box Contenuto Documento	ExAndVoteGUI	Box utile per la visualizzazione del contenuto del documento pubblicato dal team selezionato
2.11	Box Commento Attuale	ExAndVoteGUI	Box utile per: - Scrittura del commento nel caso questo fosse assente - Visualizzazione del proprio commento nel caso in cui invece questo fosse stato precedentemente assegnato al documento in questione
1.30	Indietro	TeamsGUI ←	Pulsante per tornare alla schermata precedente
1.31	Chiudi	ExAndVoteGUI	Pulsante per chiudere il pop-up di esame del documento / pubblicazione commento

HackathON! — Gestisci

Gestisci Hackathon

Aggiorna

☒ Hackathon Innovazione 2024

☐ Tecnologia

☐ iCode

Indietro Apri

#	Elemento	Classe GUI	Descrizione
1.32	Apri	→ ResumeGUI	Pulsante utile per la selezione di un hackathon di cui si è organizzatore da gestire
1.33	Aggiorna	PlannerGUI	Pulsante utile all'aggiornamento dell'elenco degli hackathon di cui l'utente è organizzatore disponibili
3.8	Hackathon Card	PlannerGUI	Card hackathon con radiobutton per la selezione di un hackathon da gestire
1.34	Indietro	HubGUI ←	Pulsante per tornare alla schermata precedente

HackathON! — Gestisci/Riepilogo

Riepilogo

— □ ×

Riepilogo Hackathon

Hackathon Innovazione 2024

Sede:

Milano

Durata:

3 giorni

Data Inizio:

2024-06-15

Data Fine:

2024-06-17

Data Apertura Iscrizioni:

2024-05-01

Data Chiusura Iscrizioni:

2024-06-10

Max Iscritti:

100

Max Dim. Team:

5

Descrizione Problema:

Sviluppare soluzioni innovative per la mobilita' sostenibile in ambito urbano

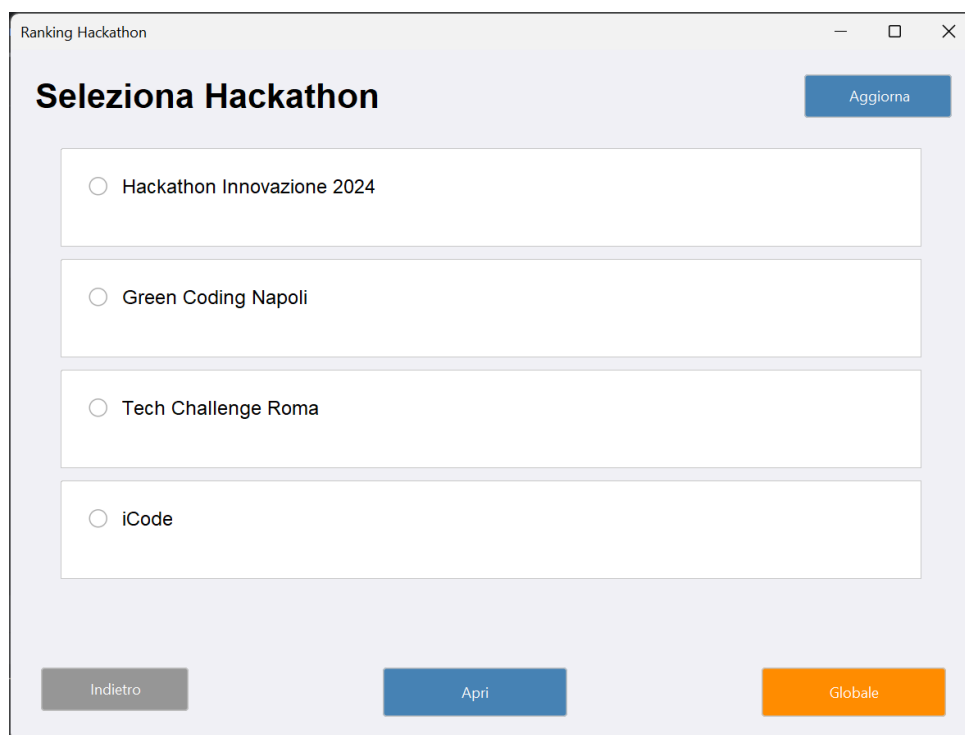
Indietro

Inizia

Termina

#	Elemento	Classe GUI	Descrizione
3.9	Campi dati Hackathon	ResumeGUI	Box per la visualizzazione degli attributi caratterizzanti ogni hackathon (Sede, Durata, Data Inizio, ...)
1.35	Inizia	ResumeGUI	Pulsante per stabilire l'inizio dell'hackathon selezionato
1.36	Termina	ResumeGUI	Pulsante per stabilire la fine dell'hackathon selezionato
1.37	Indietro	PlannerGUI ←	Pulsante per tornare alla schermata precedente

HackathON! — Classifica



Ranking Hackathon

Seleziona Hackathon

Aggiorna

☐ Hackathon Innovazione 2024

☐ Green Coding Napoli

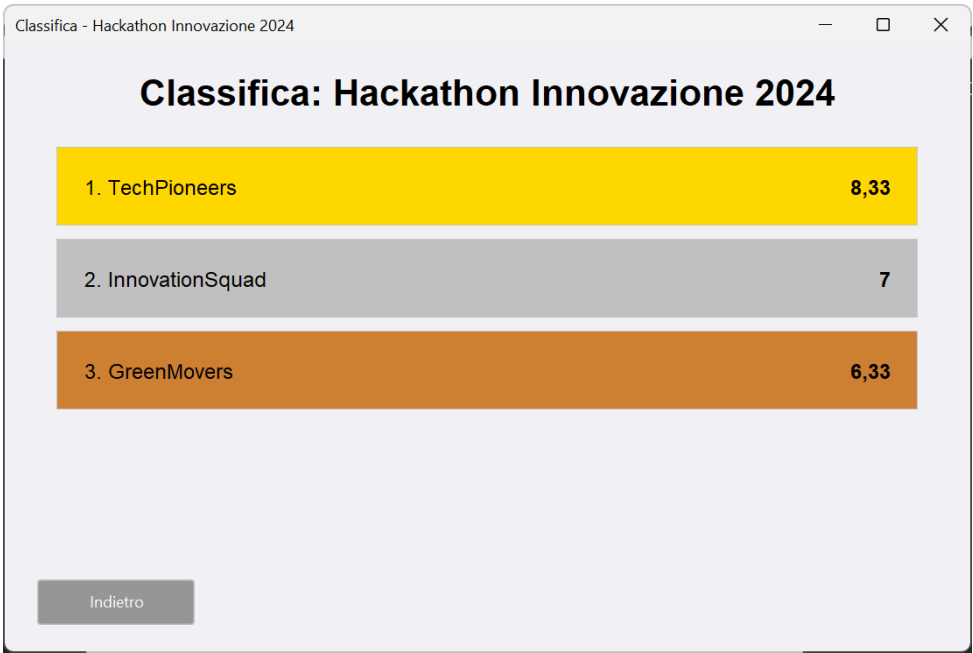
☐ Tech Challenge Roma

☐ iCode

Indietro Apri Globale

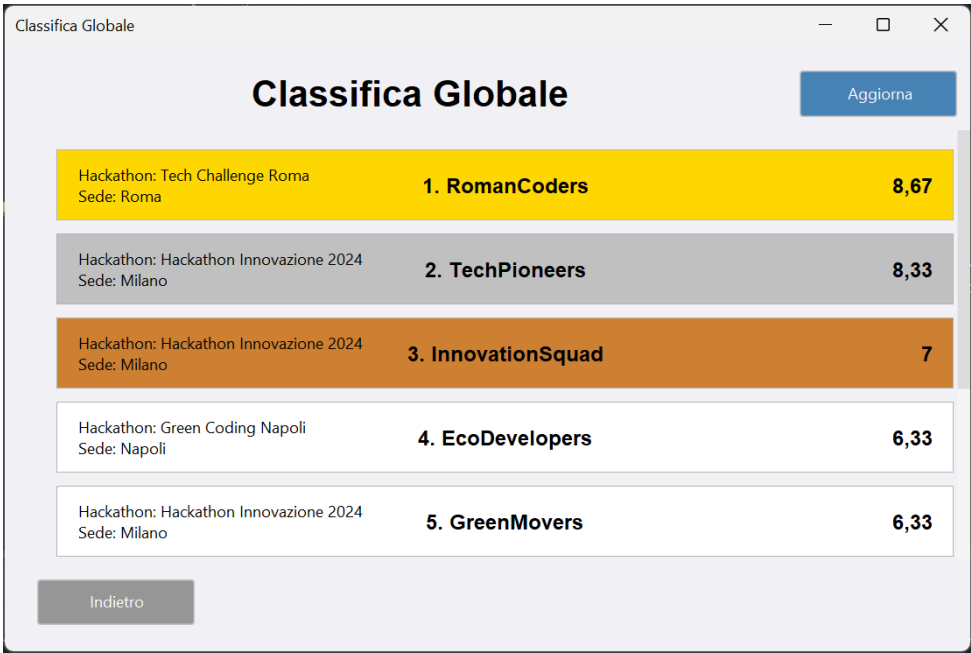
#	Elemento	Classe GUI	Descrizione
1.38	Apri	→ ScoreboardGUI	Pulsante utile per la selezione di un hackathon di cui visualizzarne la classifica dei team che vi hanno partecipato
1.39	Globale	→ ScoreboardGUI	Pulsante utile per la visualizzazione della classifica globale: elenco dei team che hanno ottenuto il maggior punteggio indipendentemente dall'hackathon in cui hanno giocato
1.40	Indietro	HubGUI ←	Pulsante per tornare alla schermata precedente
3.9	Hackathon Card	ScoreboardGUI	card con radiobutton per la selezione di un hackathon di cui visualizzarne la classifica finale dei team

HackathON! — Classifica/Apri



#	Elemento	Classe GUI	Descrizione
1.41	Aggiorna	ScoreboardGUI	Pulsante utile per l'aggiornamento dell'elenco della classifica dei team partecipanti all'hackathon selezionato
1.42	Indietro	RankingGUI ←	Pulsante per tornare alla schermata precedente
3.10	Team Card	ScoreboardGUI	card contenente nome del team componente la classifica finale dell'hackathon selezionato

HackathON! — Classifica/Globale



#	Elemento	Classe GUI	Descrizione
1.43	Aggiorna	ScoreboardGUI	Pulsante utile per l'aggiornamento dell'elenco della classifica dei team di tutti gli hackathon terminati
1.44	Indietro	RankingGUI ←	Pulsante per tornare alla schermata precedente
3.11	Card Team	ScoreboardGUI	card contenente nome del team componente la classifica globale

8 Repository GitHub

Il presente trattato è disponibile pubblicamente su GitHub al seguente link:

`github.com/TrialShock26/Hackathon`

La repository contiene:

- L'intero codice sorgente sviluppato
- La documentazione Javadoc
- Questo documento di progetto

Fine del Documento

DOCUMENTAZIONE HACKATHON

OBJECT ORIENTATION

Si ringrazia il lettore per l'attenzione

Mario Majorano	N86005035
Luca Sanselmo	N86005147

Anno Accademico 2024/2025



UNIVERSITÀ DEGLI STUDI
DI NAPOLI FEDERICO II